Project 3

Empirical Performance Comparison of Raw SQL and SQLAlchemy ORM in PostgreSQL

Execution Time, Memory Consumption, and Scalability Analysis

---

1. Introduction and Motivation

Modern backend systems frequently rely on Object-Relational Mapping (ORM) frameworks such as SQLAlchemy to simplify database interactions. ORMs abstract SQL queries into object-based operations, improving developer productivity and maintainability. However, abstraction layers may introduce performance overhead, especially in high-volume systems.

This project investigates the performance trade-offs between:

- Direct Raw SQL queries
- SQLAlchemy ORM abstraction layer

The goal was not simply to measure execution speed but to perform a systematic empirical evaluation of:

- Execution time
- PostgreSQL memory usage
- Python peak memory usage
- Scalability across dataset sizes

The motivation behind this research was to understand when abstraction improves maintainability without significantly harming performance — and when it becomes a bottleneck.

---

2. Problem Definition

The central research question:

How does SQLAlchemy ORM compare to Raw SQL in terms of execution performance and memory efficiency across different database operations and dataset sizes?

Operations tested:

1. Insert operations
2. Select operations
3. Update operations

Dataset sizes tested:

- 100 records
- 1,000 records
- 10,000 records
- 50,000 records

Each experiment was repeated three times to compute average metrics for reliability.

---

3. Experimental Environment

3.1 Database System

- PostgreSQL database
- Identical schema used for both implementations
- Separate isolated database instances
- Data cleared between test runs

This ensured no caching bias or residual state interference.

3.2 Python Environment

- Raw SQL implementation using psycopg2
- SQLAlchemy ORM implementation
- Memory profiling using Python memory tracking
- PostgreSQL memory monitoring through system tools

3.3 Data Generation

Synthetic data was generated using:

- Faker library
- Randomized transactional records
- Structured fields
- Uniform schema for fairness

This ensured controlled and reproducible dataset scaling.

---

4. Methodology

Each operation (Insert, Select, Update) was tested independently.

For each dataset size:

1. Database cleared
2. Operation executed
3. Execution time measured
4. PostgreSQL memory usage recorded
5. Python peak memory usage recorded
6. Process repeated 3 times
7. Averages computed

This controlled approach ensured statistical consistency.

---

5. INSERT Operations Analysis

5.1 Raw SQL Insert Performance

At 50,000 records:

Execution time:
~3.14 seconds

PostgreSQL memory usage:
~20.8 MB

Python memory usage:
~170 KB (stable across dataset sizes)

Observation:
Raw SQL execution time scaled efficiently and maintained stable Python memory usage.

5.2 SQLAlchemy Insert Performance

At 50,000 records:

Execution time:
~7.28 seconds

PostgreSQL memory usage:
~3.8 MB

Python peak memory usage:
~166 MB

Observation:

SQLAlchemy significantly increased Python memory usage due to:

- Object instantiation
- Session tracking
- ORM object lifecycle management

5.3 Insert Operation Conclusion

Raw SQL:

- Faster execution
- Lower Python memory usage
- Higher PostgreSQL memory consumption

SQLAlchemy:

- Slower
- Lower PostgreSQL memory
- Extremely high Python memory overhead

Conclusion:

ORM abstraction introduces significant memory overhead during bulk insert operations.

---

6. SELECT Operations Analysis

6.1 Raw SQL Selection

Small datasets:
Slightly slower than ORM

Large datasets:
Faster than ORM

PostgreSQL memory usage:
Higher than ORM

Python memory:
Moderate growth with dataset size

6.2 SQLAlchemy Selection

Small datasets:
More efficient

Large datasets:
Slightly slower but stable

PostgreSQL memory:
Lower usage compared to Raw SQL

Python memory:
Controlled growth but higher than Raw SQL for very large data

---

6.3 Selection Conclusion

SQLAlchemy performed better for small dataset retrieval due to optimized internal batching.

For large datasets, Raw SQL demonstrated better execution scaling.

This suggests:

ORM abstractions optimize convenience, not necessarily raw throughput.

---

7. UPDATE Operations Analysis

7.1 Raw SQL Update

50,000 records:

Execution time:
~399 seconds

PostgreSQL memory:
~142 MB

Python memory:
Low and stable

7.2 SQLAlchemy Update

50,000 records:

Execution time:
~199 seconds

PostgreSQL memory:
~5.6 MB

Python memory:
~6.98 MB

7.3 Update Operation Insight

Interestingly, SQLAlchemy performed better than Raw SQL in update operations at scale.

This suggests:

ORM batch handling and transaction optimization may outperform naïve Raw SQL update loops when not optimized manually.

---

8. Comparative Analysis Summary

Execution Speed

Raw SQL generally faster in Insert and large-scale Select operations.

SQLAlchemy competitive in Update operations.

PostgreSQL Memory Usage

Raw SQL consumed more PostgreSQL memory.

SQLAlchemy optimized database-level memory but increased application-layer memory.

Python Memory Usage

Raw SQL maintained low and stable memory usage.

SQLAlchemy significantly increased Python memory due to:

- ORM object instantiation
- Session tracking
- Identity map caching

---

9. Scalability Observations

As dataset size increased:

Raw SQL scaled linearly in time and memory.

SQLAlchemy showed:

- Nonlinear Python memory growth
- Higher overhead at very large datasets
- Improved structural consistency

---

10. Key Findings

1. Raw SQL is more suitable for:

    - High-performance systems
    - Memory-constrained environments
    - Bulk data operations
    - Real-time systems
2. SQLAlchemy is more suitable for:
    - Complex business logic
    - Maintainability-focused systems
    - Rapid development environments
    - Medium-scale applications

---

11. Technical Skills Demonstrated

- PostgreSQL performance analysis
- Memory profiling
- ORM internals understanding
- Backend benchmarking methodology
- Experimental design
- Statistical averaging
- Python backend development
- Database schema control
- Controlled environment testing

---

12. Limitations

- No distributed database testing
- No multi-threaded concurrency benchmarking
- No query indexing optimization variations
- Focus limited to single-node PostgreSQL

---

13. Real-World Implications

This study informs backend architectural decisions in:

- High-frequency trading systems
- Real-time analytics platforms
- Enterprise backend services
- API-heavy microservices

- Scalable SaaS applications

It demonstrates that abstraction convenience must be evaluated against performance cost.

---

14. Future Improvements

- Test under concurrent workloads
- Include indexing optimization
- Compare with other ORMs (Django ORM, Hibernate)
- Benchmark under distributed database setups
- Include connection pooling experiments

This is your full detailed documentation for Project 3.

Next project will be:

SkillSwap – Full Stack Application

Say:

NEXT

And we continue.